

ATOB / TrackTransporter — სრული ბიზნეს ლოგიკა

საგზაო ტვირთგადაზიდვის მართვის სისტემა (Road Freight SaaS) — კოდის ანაზღაურებადი დოკუმენტი **Stack:** Spring Boot + PostgreSQL (Flyway) · React Native (Expo) · HTTP Basic Auth

1. რა არის აპლიკაცია

საგზაო ტვირთგადაზიდვის (road freight) მართვის სისტემა — Amazon Relay-ის სტილის.

ნაკადი:

- მომხმარებელი (Customer)** ათვალიერებს პროდუქტებს და აკეთებს ტრანსპორტირების შეკვეთას
- ადმინი (Admin)** ამუშავებს შეკვეთას ეტაპობრივად და ანიჭებს მძღოლს
- მძღოლი (Driver/Carrier)** ფიზიკურად აწვდის ტვირთს და track-ავს ლოკაციას GPS-ით
- შეკვეთის გადაცემისას **ავტომატურად იხსნება ჩატი** მომხმარებელსა და მძღოლს შორის

2. მონაწილეები (Roles)

სამი როლი, ინახება UserRole-ში ROLE_ პრეფიქსით:

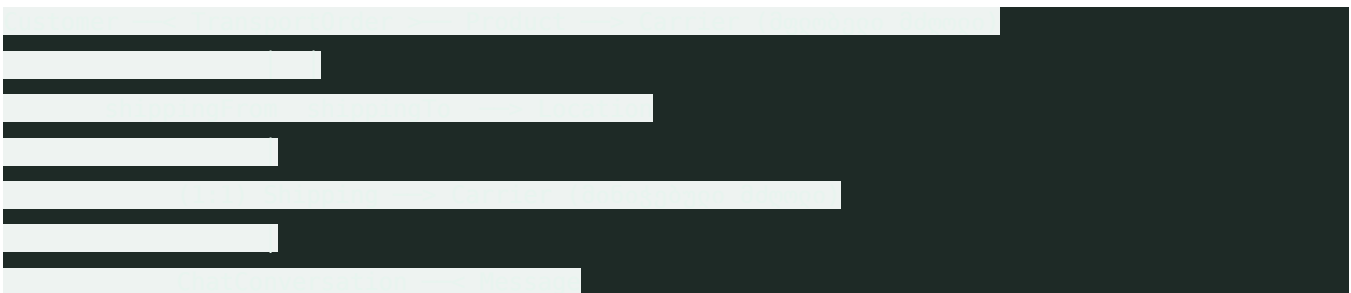
როლი	ვინ არის	ძირითადი უფლებები
ADMIN	ოპერატორი/დისპეტჩერი	ყველა write-ოპერაცია: სტატუსები, მძღოლის მინიჭება, პროდუქტი/ლოკაცია, სტატისტიკა
CUSTOMER	ტვირთის მფლობელი	შეკვეთის შექმნა, საკუთარი შეკვეთების ნახვა, ჩატი
DRIVER	მძღოლი	shipping-ის pickup/deliver, GPS tracking, ჩატი

როლი	ვინ არის	ძირითადი უფლებები
------	----------	-------------------

(Carrier)

ავთენტიფიკაცია: Spring Security HTTP Basic. `ServiceUser` — ცალკე entity credentials-ისთვის (`username=email`, `password={noop}plaintext` ან `{bcrypt}hash`). Customer/Carrier ცალკე ბიზნეს-entity-ებია, დაკავშირებული **email-ით** (არა FK-ით).

3. დოკუმენტის მოდელი



Entity	აღწერა
TransportOrder	შეკვეთა: <code>OrderStatus</code> , customer, product, from/to, orderDate, shippingDate, deliveredDate
Shipping	ფიზიკური მიწოდება (1:1 order-თან, <code>order_id</code> UNIQUE): <code>CarrierStatus</code> , carrier, GPS <code>numeric(50,20)</code>
Product	მარაგი (<code>stock=BigInteger</code>), სურათები BYTEA (<code>image1</code> აქტიური, 2-6 რეზერვი), <code>imageUrl</code> fallback
Customer / Carrier	id, email, firstName, lastName, password, address, postcode, city, phone (postcode/phone = String)
Location	id, address, postcode (Integer), city, phone (Integer)
ChatConversation	ერთ shipping-ზე ერთი ჩატი (<code>shipping_id</code> UNIQUE), customerId, driverId, active
Message	senderId, senderRole (CUSTOMER/DRIVER), content (≤ 2000), sentAt,

read

4. რეგისტრაცია & ავტორიზაცია

RegistrationService.java — **Customer / Driver signup** (ერთი ლოგიკა):

- ვალიდაცია: email უნდა შეიცავდეს @; password აუცილებელია
- დუბლიკატის შემოწმება email-ით
- იქმნება ბიზნეს-entity (Customer/Carrier) UUID-ით
- პარალელურად** იქმნება ServiceUser (login) + UserRole: - Customer → ROLE_CUSTOMER - Driver → ROLE_DRIVER
- პაროლი ინახება {noop} პრეფიქსით (plaintext — dev რეჟიმი)

ADMIN ჩაშენებულია seed-ში ({bcrypt}); რეგისტრაციით ვერ შექმნი. /auth/me (AuthController) — აბრუნებს role + customerId/carrierId-ს frontend-ისთვის.

⚠ **რისკი:** updateCustomer პაროლს {noop}-ის გარეშე ინახავს → შემდეგი login დაიმტკრევა.

5. შეკვეთის სასიცოცხლო ციკლი (State Machine)

OrderService.java — OrderStatus: Pending → Processing → WaitingCarrier → Shipped → Delivered (+ Cancelled). გადასვლები **შეკვეთის** კონტროლდება — ეტაპს ვერ გადახტები:



1. შექმნა — `newOrders()` — **POST** /show/order/{customerId} - UUID, OrderStatus.Pending, orderDate = დღეს; Customer უნდა არსებობდეს - **Stock ვალიდაცია:** პროდუქტი უნდა იყოს მარაგში (> 0) → წარმატებისას **-1**, თუ არა → "out of stock" - shippingDate წარსულში ვერ იქნება

2. რედაქტირება — `updateOrder()` — მხოლოდ Pending; თარიღების ვალიდაცია (shipping $\geq$

order, delivered $\geq$ shipping)

3. სტატუსის გადასვლები (თითო ამონაშენს წინა სტატუსს):

გადასვლა	პირობა	side-effect
→ Processing	Pending-დან	—
→ WaitingCarrier	Processing-დან	ავტომატურად იქმნება Shipping (Assigned, deliveryStartAt=დღეს)
→ Shipped	WaitingCarrier-დან	Shipping + carrier უნდა იყოს ; shippingDate=დღეს
→ Delivered	Shipped-დან	deliveredDate=დღეს

4. გაუქმება — `cancelOrder()` — მხოლოდ Pending; მარაგი ბრუნდება +1

კითხვა/ძებნა: `/show/orders?status=` · `/show/orders/search?q=` · `/show/orders/my` · `/show/stats` (ADMIN)

6. Shipping & მძღოლის ლოგიკა (backend)

ShippingService.java — `CarrierStatus`: Assigned → PickedUp → Delivered.

მძღოლის მინიჭება — `assignCarrier()` (@Transactional, ADMIN): 1. Shipping და Carrier უნდა არსებობდეს 2. ერთი მძღოლი = ერთი აქტიური მიწოდება: თუ მძღოლს აქვს არა-Delivered shipment → "already assigned to an active shipment" 3. ენიჭება carrier, Assigned 4.

ავტომატურად იქმნება **ChatConversation** (customer ↔ driver)

- **Pickup** — carrier მინიჭებული + Assigned → PickedUp
- **Deliver** — PickedUp → Delivered
- **GPS Tracking** — PUT `/shipping/tracking/{id}/{lat}/{lng}`
- **updateShipping** — მასობრივი update მხოლოდ PickedUp-ზე

Order-ისა და Shipping-ის სტატუსები ცალ-ცალკეა და ხელით უნდა იყოს სინქრონიზებული.

7. ჩატის სისტემა

ChatServiceImpl.java

- **მონაწილის ამოცნობა (resolve):** email-ით ვადაგენთ Customer-ია თუ Carrier; ვერცერთი → `ForbiddenException`
- **წვდომა:** მომხმარებელი ხედავს მხოლოდ თავის conversation-ებს; უცხო ჩატზე → 403
- **sendMessage:** არა ცარიელი, max 2000 სიმბ.; ანახლებს `updatedAt`-ს
- **წაკითხვა:** ჩატვირთვისას ავტომატურად `read=true` (გამგზავნის გარდა); `unreadCount` თვლის წაუკითხავებს
- **იდემპოტენტური შექმნა:** raw JDBC `INSERT ... ON CONFLICT (shipping_id) DO NOTHING` — race-safe, ერთ shipping-ზე ერთი ჩატი

8. ავტორიზაციის მატრიცა

CustomWebSecurityConfiguration.java — HTTP Basic, CSRF off, CORS ღია (*).

Endpoint	წვდომა
<code>/register/signUp*</code>	public
GET <code>/products/*/image</code>	public (UUID-ით)
<code>/auth/me, /chat/**</code>	ნებისმიერი authenticated
POST <code>/show/order/**</code>	ADMIN, CUSTOMER
GET <code>/show/orders/my</code>	CUSTOMER
GET <code>/show/order(s)/**, /shipping/**</code>	ADMIN, DRIVER
GET <code>/show/stats, /register/customers</code>	ADMIN only
PUT <code>/shipping/*/pickup, /deliver, /tracking/**</code>	ADMIN, DRIVER
სხვა POST/PUT/DELETE (show, products, locations, assign, register)	ADMIN only

9. დამხმარე entity-ები

- **Product** (ProductController): CRUD (ADMIN), stock ნეგატიური ვერ იქნება, სურათის upload ($\leq 15\text{MB}$) + public serve (cache 24სთ)
- **Location**: CRUD ლოკაციებზე (from/to მისამართები)

10. ვალიდაცია & შეცდომები

ExceptionHandlerAdvice.java — ერთიანი ApiErrorDTO (status/error/message/path/timestamp):

Exception	HTTP	მნიშვნელობა
BadRequestException	400	ბიზნეს-წესის დარღვევა
ForbiddenException	403	ჩათის წვდომა
სხვა Exception	500	message იმალება, ლოგში სრული stack

11. Frontend ლოგიკა (React Native / Expo)

საქაღალდე: frontend/src/ (atob-frontend). ფაილები .js (არა TypeScript). App.js → LanguageProvider → NavigationContainer → AppNavigator.

11.1 API ფენა — api/client.js + config.js

- Axios instance, baseUrl = API_BASE_URL, timeout 10წმ
- config.js-ში API_BASE_URL ამჟამად **ngrok tunnel URL**-ია (https://...ngrok-free.app) — single source of truth; tunnel-ის restart-ზე იცვლება ერთ ხაზში, prod-ში → https://your-domain.com
- ყველა request-ს ემატება header ngrok-skip-browser-warning: true
- **request interceptor** — AsyncStorage-დან იღებს authToken-ს და ურთავს Authorization: Basic <token>
- **response interceptor** — 401-ზე ასუფთავებს credential-ებს (authToken, userRole, carrierId, customerId, ...)
- login() — RN-safe **custom base64** encoder → ინახავს authToken-ს → GET /auth/me

- API ჯგუფები: **Orders** (/show/...), **Shippings** (/api/shippings/...), **Products, Locations, Customers/Drivers, Registration** (public, raw axios), **Chat** (/chat/...)

11.2 ავთენტიფიკაცია — App.js (Context-ის გარეშე)

- AppNavigator გაშვებისას კითხულობს AsyncStorage-ს ['authToken', 'userRole'] → ირჩევს initial route-ს: Login / Main (admin) / DriverMain / CustomerMain
- LoginScreen: login() + getMe() → იწახვას userRole, carrierId, customerId, firstName, lastName → navigation.replace(dest) როლის მიხედვით
- logout: AsyncStorage.multiRemove([...])

11.3 მრავალენოვნება (i18n) — i18n/index.js

- **17 ენა:** ka (ნაგულისხმევი), en, ru, de, tr, fr, es, ar, zh, it, pt, uk, pl, nl, ro, ja, ko
- LanguageProvider + useTranslation() → t('a.b.c') წერტილოვანი key-ებით; key ვერ მოიძებნა → თავად key ბრუნდება
- არჩეული ენა იწახვება AsyncStorage-ში (appLang)
- ენის გადართვა: SettingsScreen (სრული სია)

11.4 როლზე დაფუძნებული ნავიგაცია — App.js

ერთი root Stack, თითო როლს ცალკე Bottom-Tab ხე აქვს:

როლი	Tabs	სტეკები
არა-ავტორიზებული	—	Login → RegisterType → RegisterForm
ADMIN (Main)	Dashboard · Orders · Products · Drivers · Settings	Orders→(OrdersList, OrderDetail, GPSTracking, CreateOrder); Products→(ProductsList, ProductForm)
DRIVER (DriverMain)	DriverHome · Messages · Settings	Chat→(ConversationList, ChatThread)
CUSTOMER (CustomerMain)	Shop · MyOrders · Messages · Settings	Shop→(CustomerShop, CustomerOrderForm, CustomerOrders); Chat→(ConversationList, ChatThread)

11.5 ძირითადი ეკრანები

ეკრანი	ლოგიკა
LoginScreen	username/password → login+/auth/me → role-ით redirect; register link
RegisterType → RegisterForm	customer/driver არჩევა → 8-ველიანი ფორმა → signUpCustomer/Driver (public) → Login; ამუშავებს "already exists"
DashboardScreen (ADMIN)	getStats() + getOrders() → 7 stat-ბარათი + ბოლო შეკვეთები; pull-to-refresh
OrdersListScreen (ADMIN/DRIVER)	სტატუს-ფილტრის chip-ები + debounced ძებნა; FAB→CreateOrder (admin); → OrderDetail
OrderDetailScreen	ACTION_KEYS state machine (Pending→processing/cancel · Processing→waitingCarrier · WaitingCarrier→shipped · Shipped→delivered); მძღოლის მინიჭების modal (getDrivers→assignCarrier); Track GPS; confirm dialog
DriverHomeScreen (DRIVER)	getAllShippings() → აქტიური shipment; Leaflet რუკა (WebView) ; ნამდვილი GPS (expo-location) → updateCoordinates; გადასვლები startPickup→pickedUp→inTransit→deliver; auto-refresh 15წმ აქტიურზე
GPSTrackingScreen	Leaflet WebView რუკა; "ჩემი ლოკაცია" (expo-location); იგივე გადასვლები; default თბილისი (41.6938, 44.8015)
CustomerShopScreen (CUSTOMER)	getProducts() ბადედ, სურათი /products/{id}/image; out-of-stock დაბლოკილი; tap → CustomerOrderForm
CustomerOrderFormScreen	getLocations() → from/to + date → createOrder(customerId, {product, shippingFrom, shippingTo})
CustomerOrdersScreen	getMyOrders() (/show/orders/my)
CreateOrderScreen (ADMIN)	4-ნაბიჯიანი wizard: customer → product → route →

	confirm → createOrder
ProductsScreen / ProductFormScreen (ADMIN)	CRUD + სურათის upload (expo-image-picker: გალერეა/კამერა)
DriversScreen (ADMIN)	getDrivers() სია (avatar, contact)
ChatConversationListScreen / ChatContactsScreen	getChatConversations(); avatar, ბოლო მესიჯი, unread badge → ChatThread
ChatThreadScreen	polling 566 ; sendMessage; bubble mine/theirs; auto-scroll; maxLength 1000
StatusBadge (component)	ფერადი ჭდე: ORDER + SHIPPING (extended) + legacy carrier mapping

11.6 Frontend ↔ Backend ()

- Shipping API:** frontend იძახებს /api/shippings/**-ს გაფართოებული სტატუსებით (CREATED, ASSIGNED, PICKUP_IN_PROGRESS, PICKED_UP, IN_TRANSIT, DELIVERED, CANCELLED, FAILED), მაგრამ /home/python/src-ის backend აწვდის /shipping/**-ს და მხოლოდ Assigned/PickedUp/Delivered-ს. ე.ი. frontend უფრო **მონინავე shipping სერვისს** მოელის.
- config:** API_BASE_URL dev ngrok tunnel-ზეა მიბმული (არა მუდმივ prod URL-ზე).
- ProductsScreen** იყენებს hard-coded http://localhost:8080-ს config-ის ნაცვლად (მცირე შეუსაბამობა).
- საერთო შაბლონი:** useFocusEffect(load) + api + t() + StatusBadge; real-time = **polling** (არა WebSocket); single source of truth = **backend**.

12. არქიტექტურის მიმოხილვა (ფენები)

Controller (controller/)	HTTP endpoints; უსაფრთხოება SecurityConfig-შია
Service (service/)	ბიზნეს წესები, ვალიდაცია, state machine

Repository (repository/) Spring Data JPA

Domain (domain/) JPA entity-ები (Lombok)

DTO (dto/) chat request/response (entity არ ჟონავს)

Exceptions (exceptions/) BadRequestException, ForbiddenException

DB: PostgreSQL · Flyway მართავს schema-ს (V1__init_schema.sql) · Hibernate ddl-auto=validate
· dev seed R__seed_test_data.sql (repeatable, TRUNCATE CASCADE) · prod-ში seed არ იტვირთება · naming camelCase→snake_case.

შემაჯამებელი ბიზნეს-წესები

1. შეკვეთა მკაცრი **state machine**-ია — ეტაპს ვერ გადახტები
2. **Stock** იკლებს შეკვეთისას, ბრუნდება გაუქმებისას
3. **Shipped** მხოლოდ მინიჭებული მძღოლით
4. ერთი მძღოლი — ერთი აქტიური მიწოდება
5. **carrier**-ის მინიჭება ავტომატურად ხსნის ჩატს
6. ერთ **shipping**-ზე — ერთი ჩატი (idempotent)
7. ჩატი მხოლოდ მონაწილეებისთვის
8. რედაქტირება მხოლოდ **Pending / PickedUp** სტატუსებზე
9. **frontend** — როლზე დაფუძნებული (3 ცალკე ნავიგაცია) + **17-ენიანი i18n**